

Introduction to Computer Architecture

Chapter 4 - 6

Pipeline Hazards 2

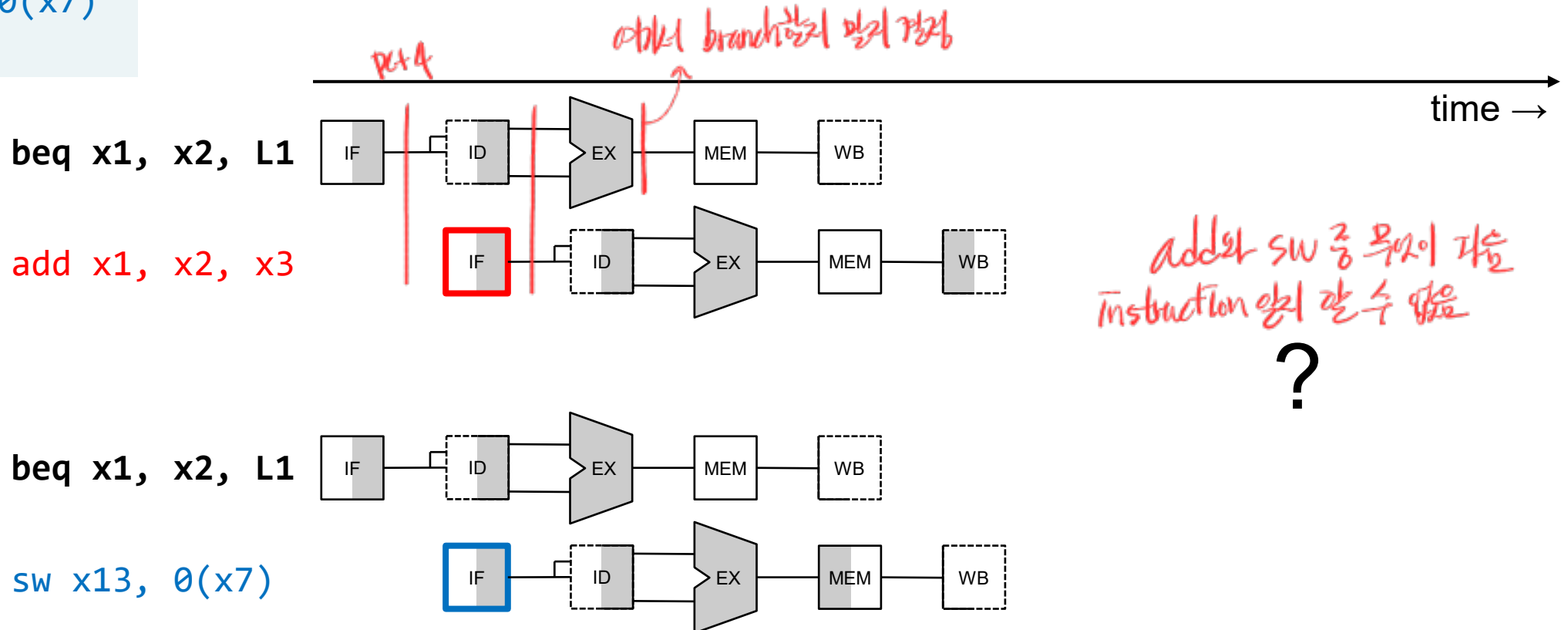
Hyungmin Cho

Department of Computer Science and Engineering
Sungkyunkwan University

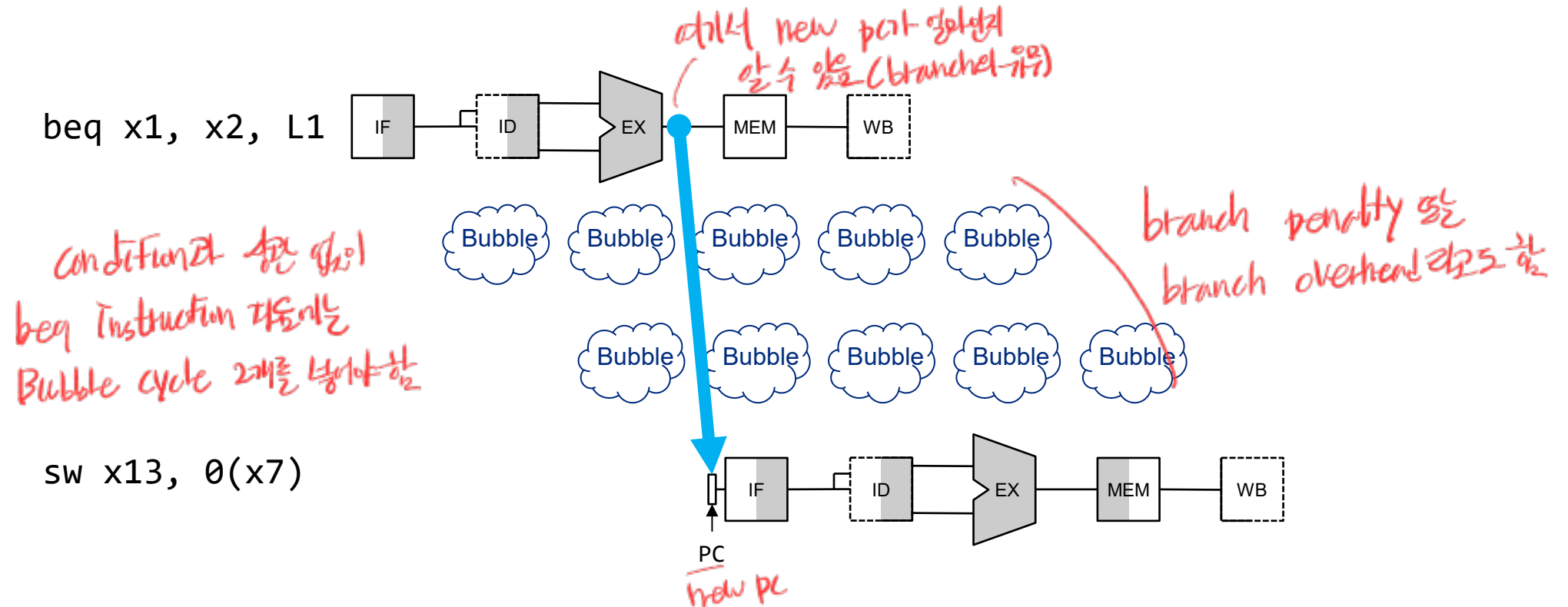
Branch Hazards

Control hazard

```
beq x1, x2, L1
add x1, x2, x3
...
...
L1: sw x13, 0(x7)
```



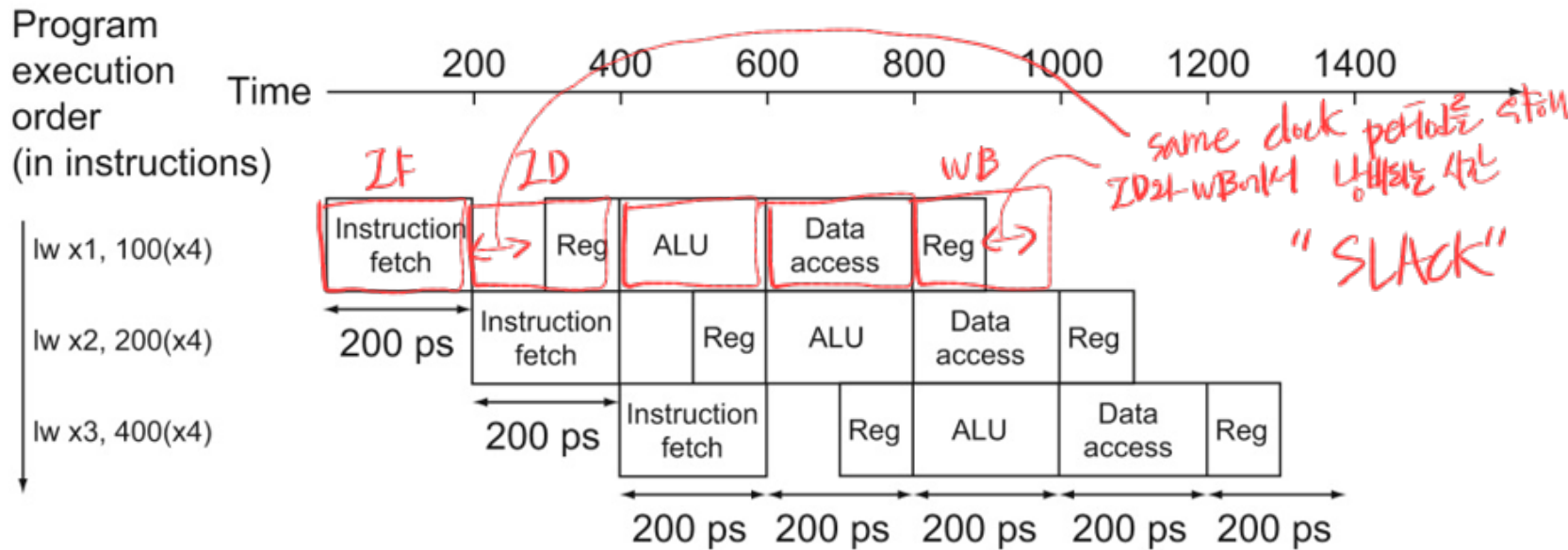
Branch Hazards



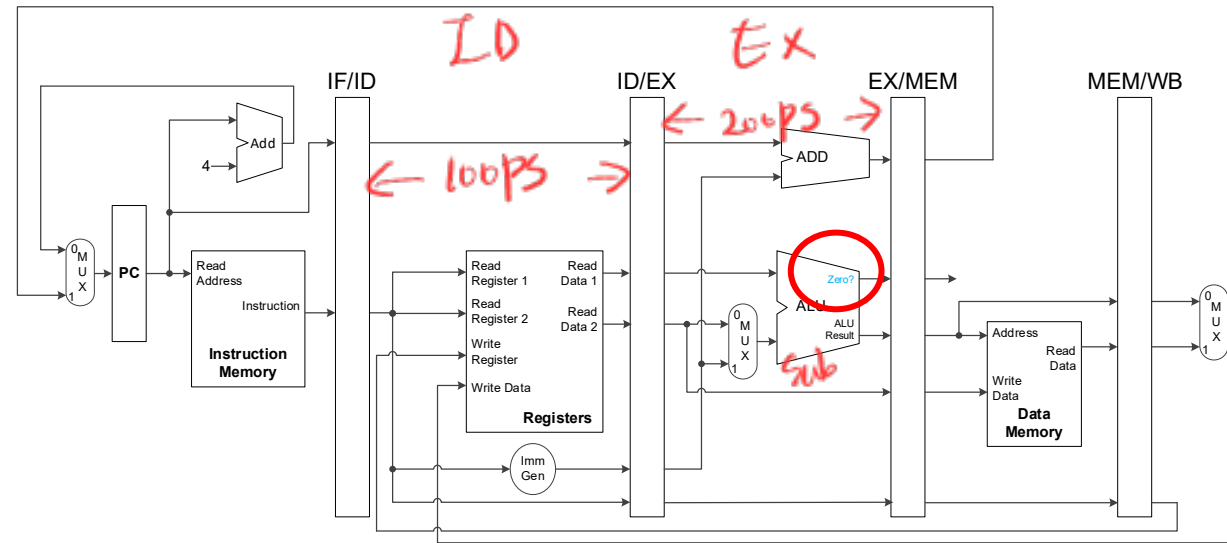
beq와 bubble cycle $\rightarrow$ performance에 영향 (beq and bubble cycle $\rightarrow$ effect on performance)

Reducing Branch Delay

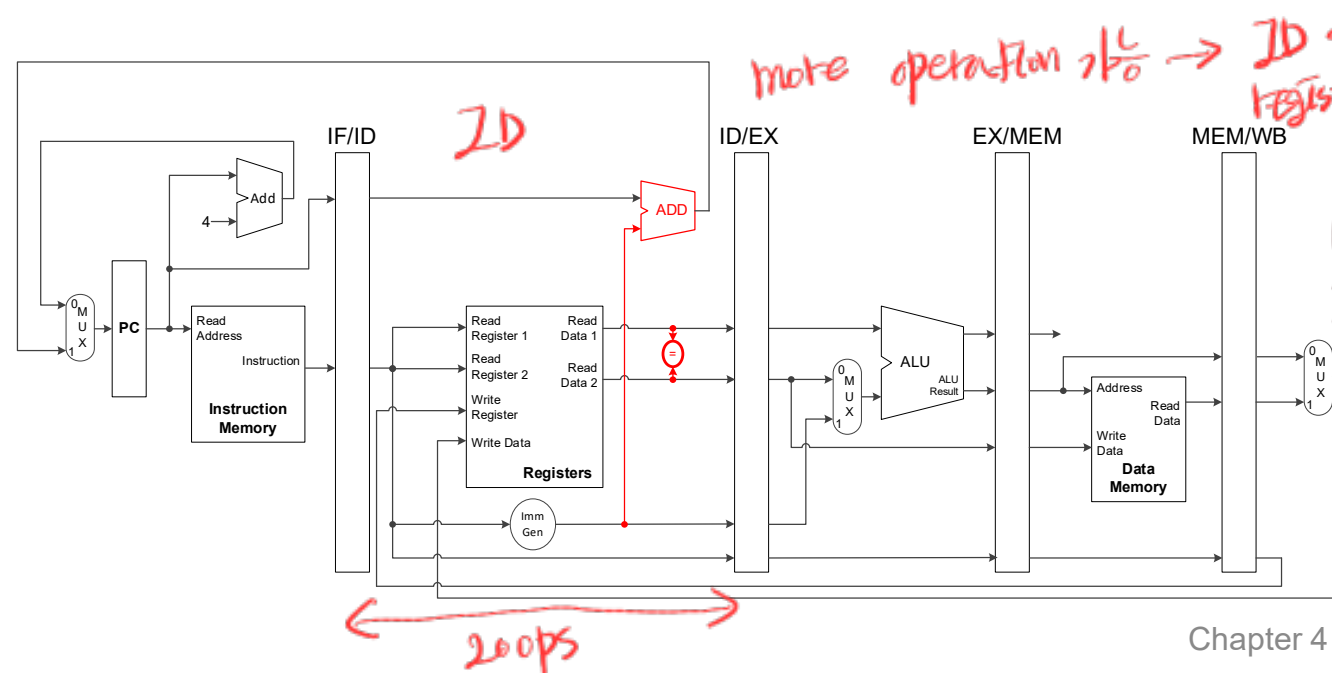
- Move register comparison to the ID stage
- Move target address calculation to the ID stage



Reducing Branch Delay



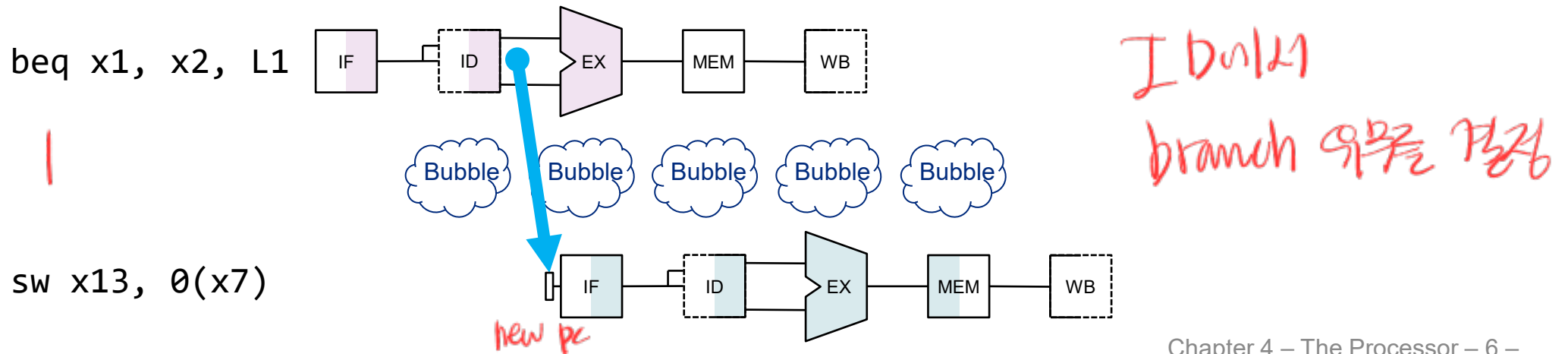
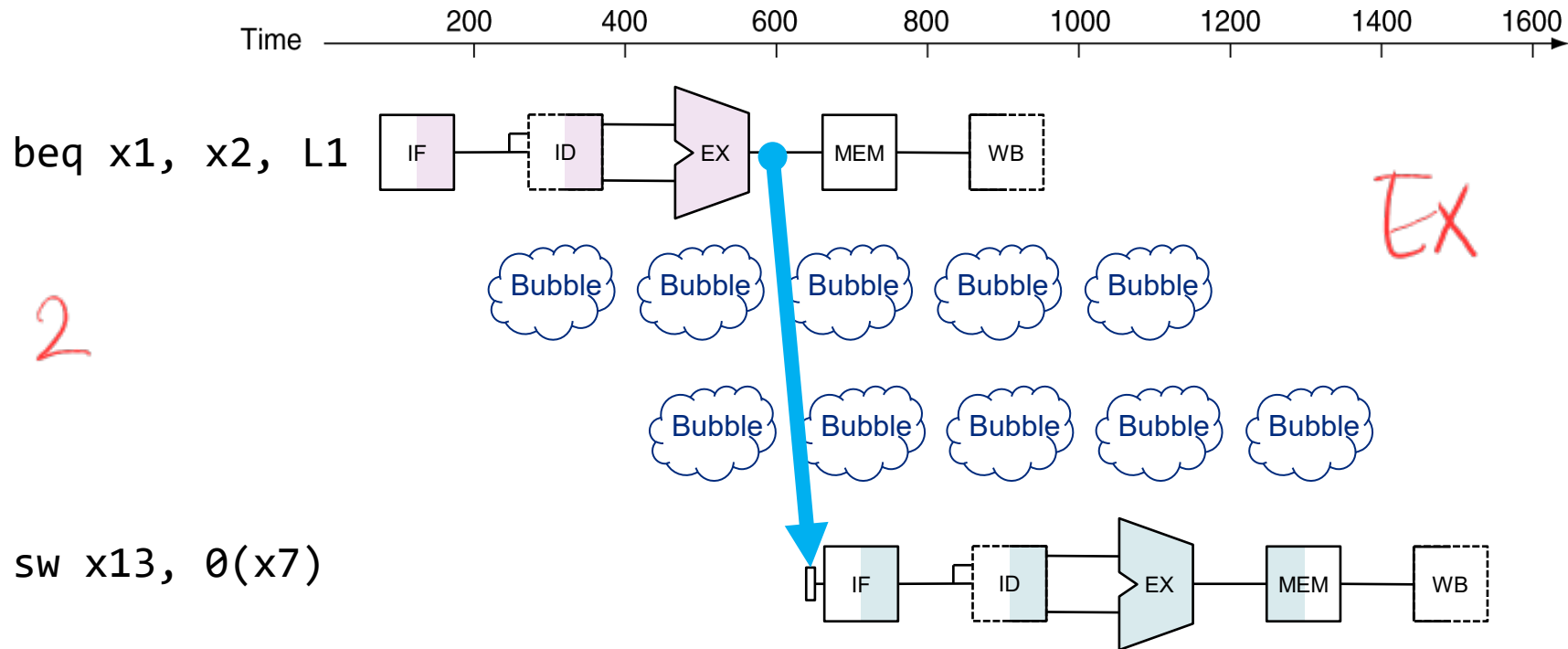
beq x1, x2, L1
 ↗
 Subtract operation 필요



more operation time -> ID stage에서 register의 값을 바꾼다

EX stage 끝난 뒤
 ID stage에서
 branch 여부를 알 수 있음

Reducing Branch Delay



Data Hazards for Branches

lw x8, ...

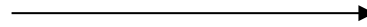
add x6, x5, x6

beq x1, x4, target

bubble

bubble

PC+4 or Target



lw x8, ...

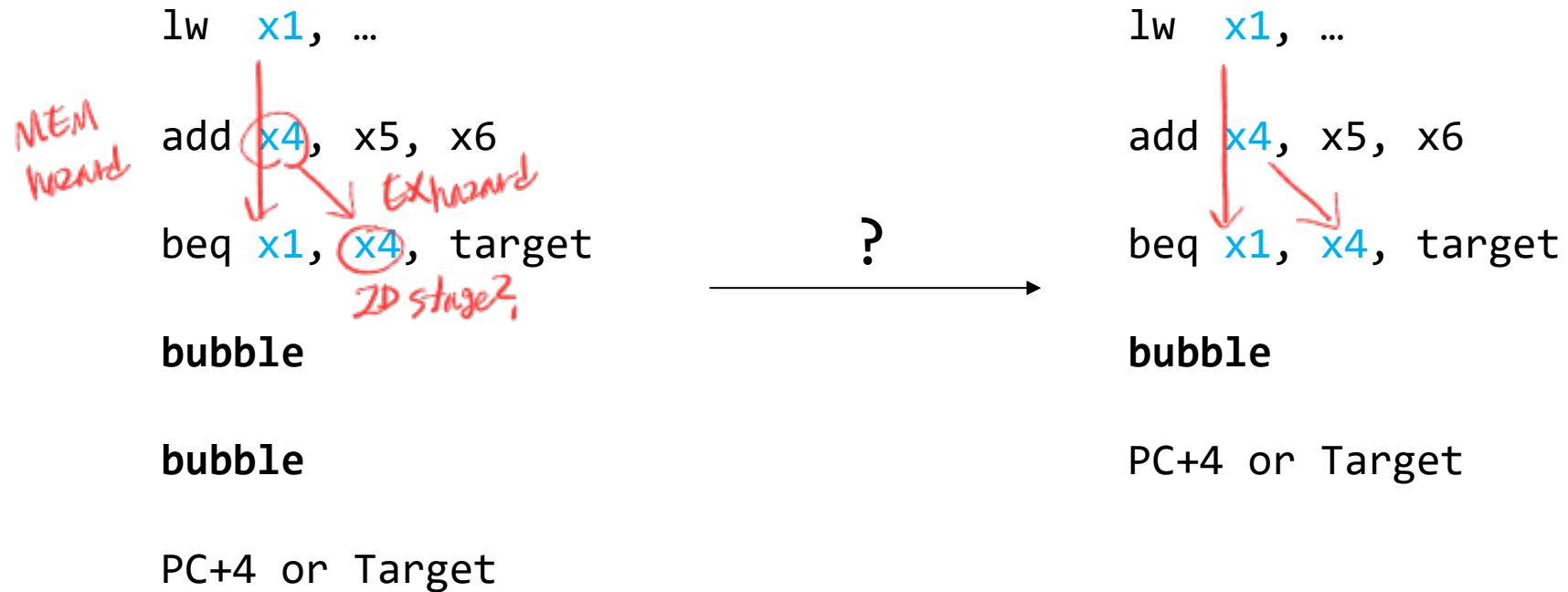
add x6, x5, x6

beq x1, x4, target

bubble

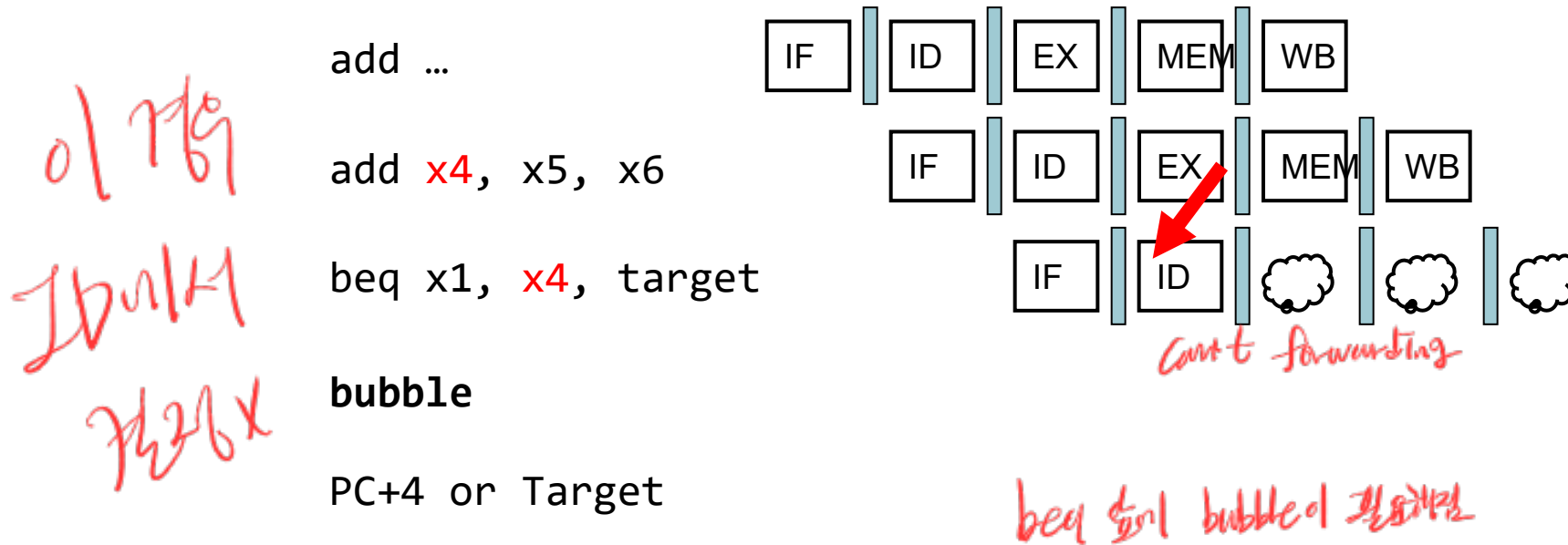
PC+4 or Target

Data Hazards for Branches



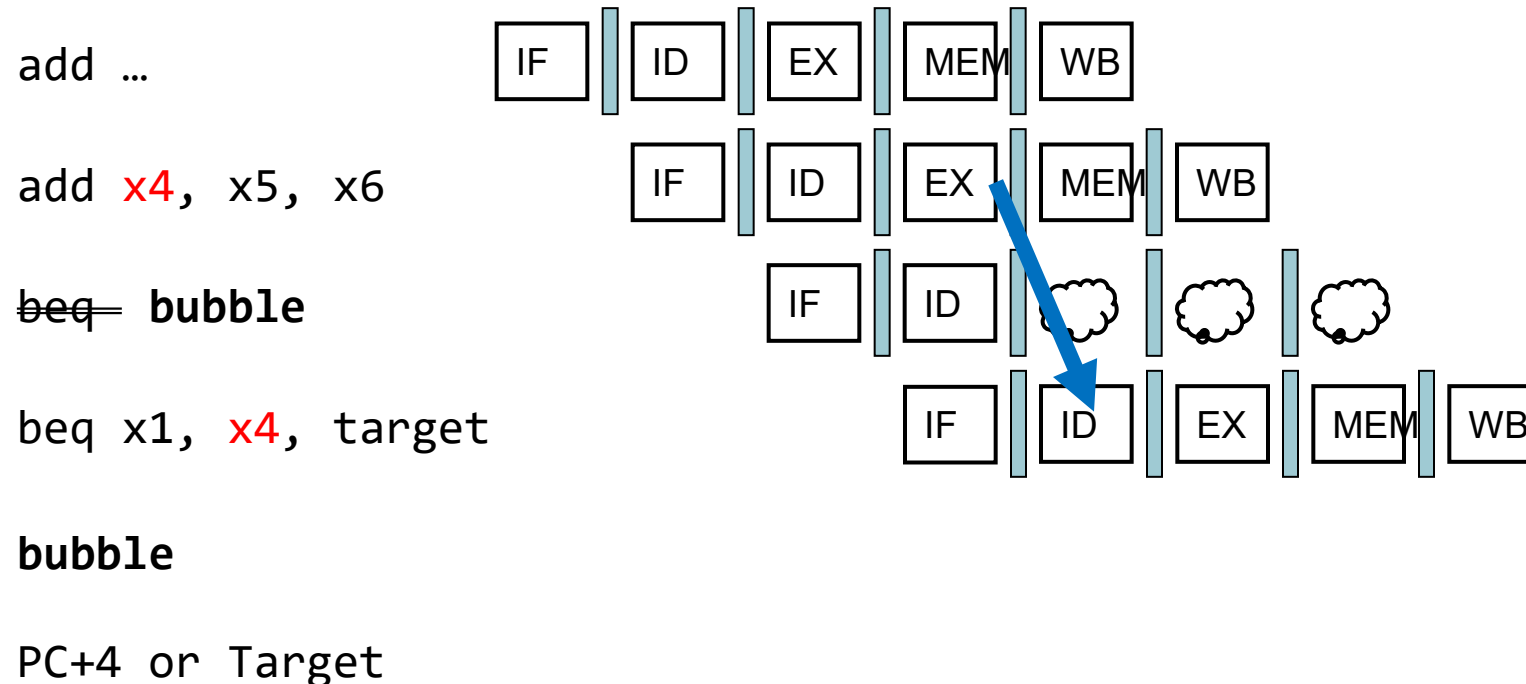
Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
 - ❖ Need 1 stall cycle BEFORE beq



Data Hazards for Branches

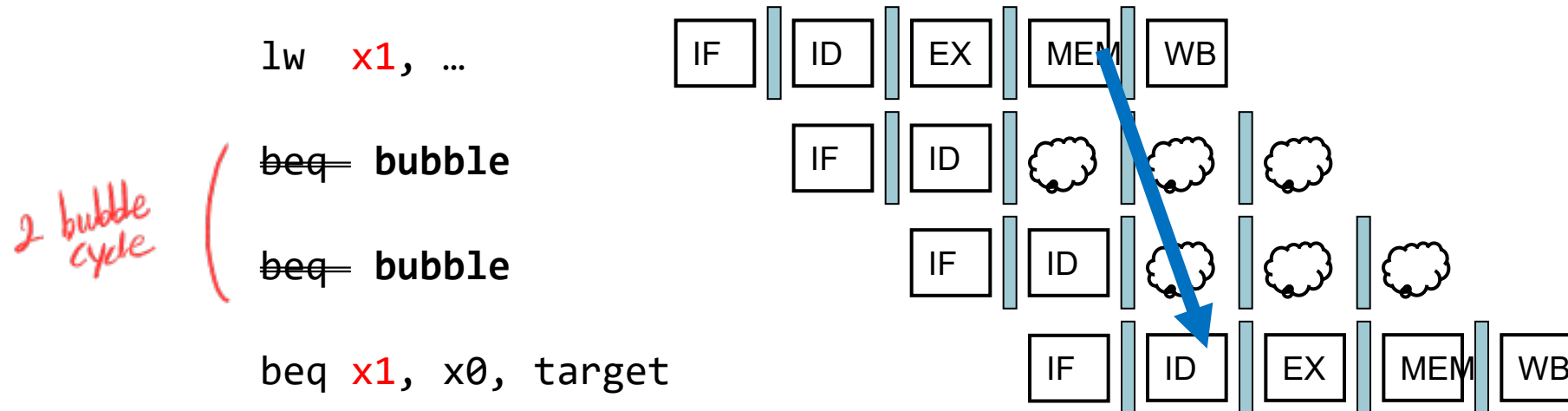
- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
 - ❖ Need 1 stall cycle BEFORE beq



Data Hazards for Branches

- If a comparison register is a destination of immediately preceding load instruction
 - ❖ Need 2 stall cycles BEFORE beq

파란색으로 ID에서 정지



EX에서 203 순서는
3 cycle이 필요한 것

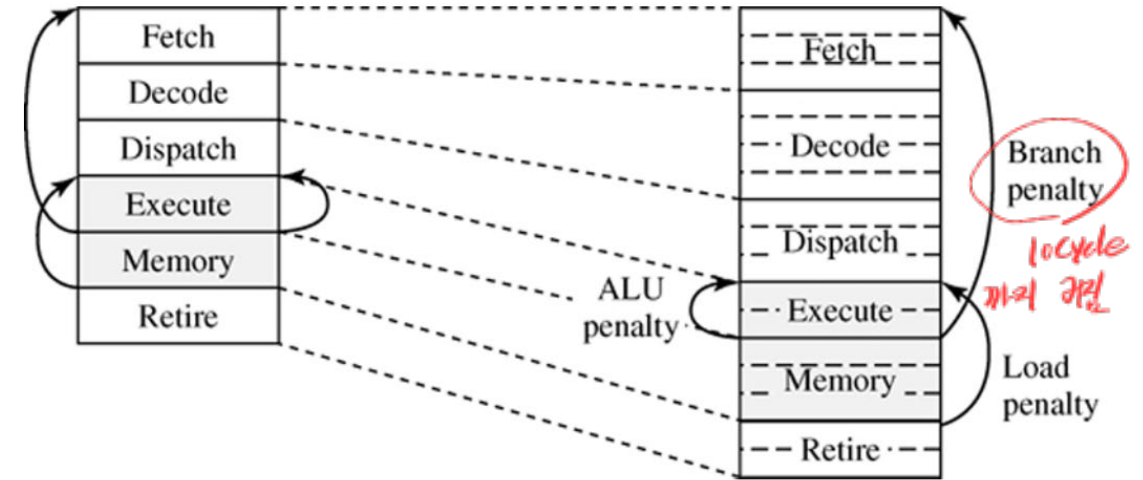
branch가 더 빨리 결정하게 해
branch penalty를 줄여줬어
순서가 중요.

Branch Prediction

- Longer pipelines can't determine branch outcome early

- ❖ Intel i7: 14 pipeline stages
- ❖ Stall penalty becomes unacceptable

branch penalty가 심해짐



- Predict the outcome of the branch instructions

- ❖ Stall only when the prediction was incorrect

예상이 틀린 경우 stall(bubble)

Branch Prediction

branch을 처리할 target을 calculate

■ An easy solution

❖ simple Always predict branches as not taken

- Proceed to PC+4

① branch을 하느냐? ← 이걸 ①이 결정

② 이걸 branch하느냐?

↑ Always prediction
↓ Always predict as taken

→ branch target은 4만큼 pc+4

Taken branch → Jump to Label 1

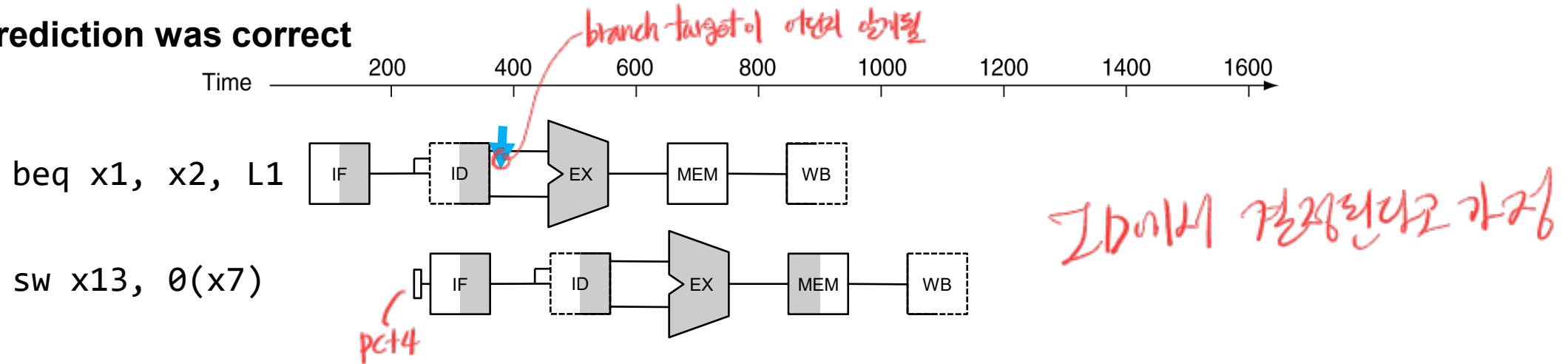
Not Taken branch → PC+4

❖ If the prediction was incorrect,

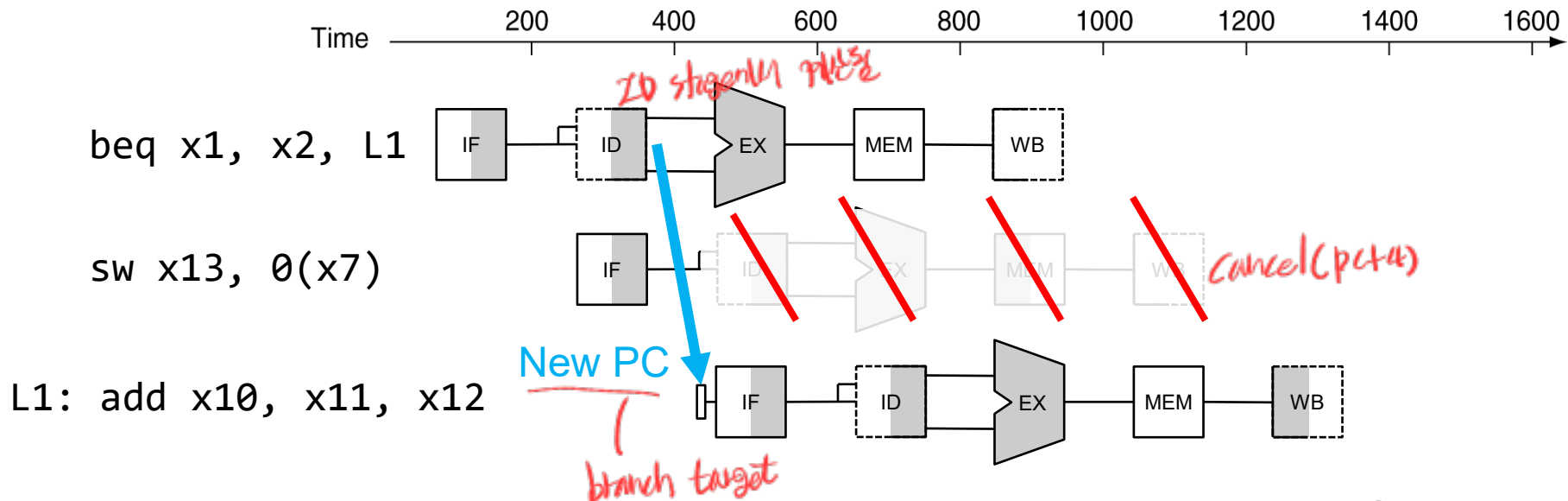
- Cancel the predicted instructions in the pipeline.
- Reset the PC value with the true target of the branch

Predict Not Taken

Prediction was correct



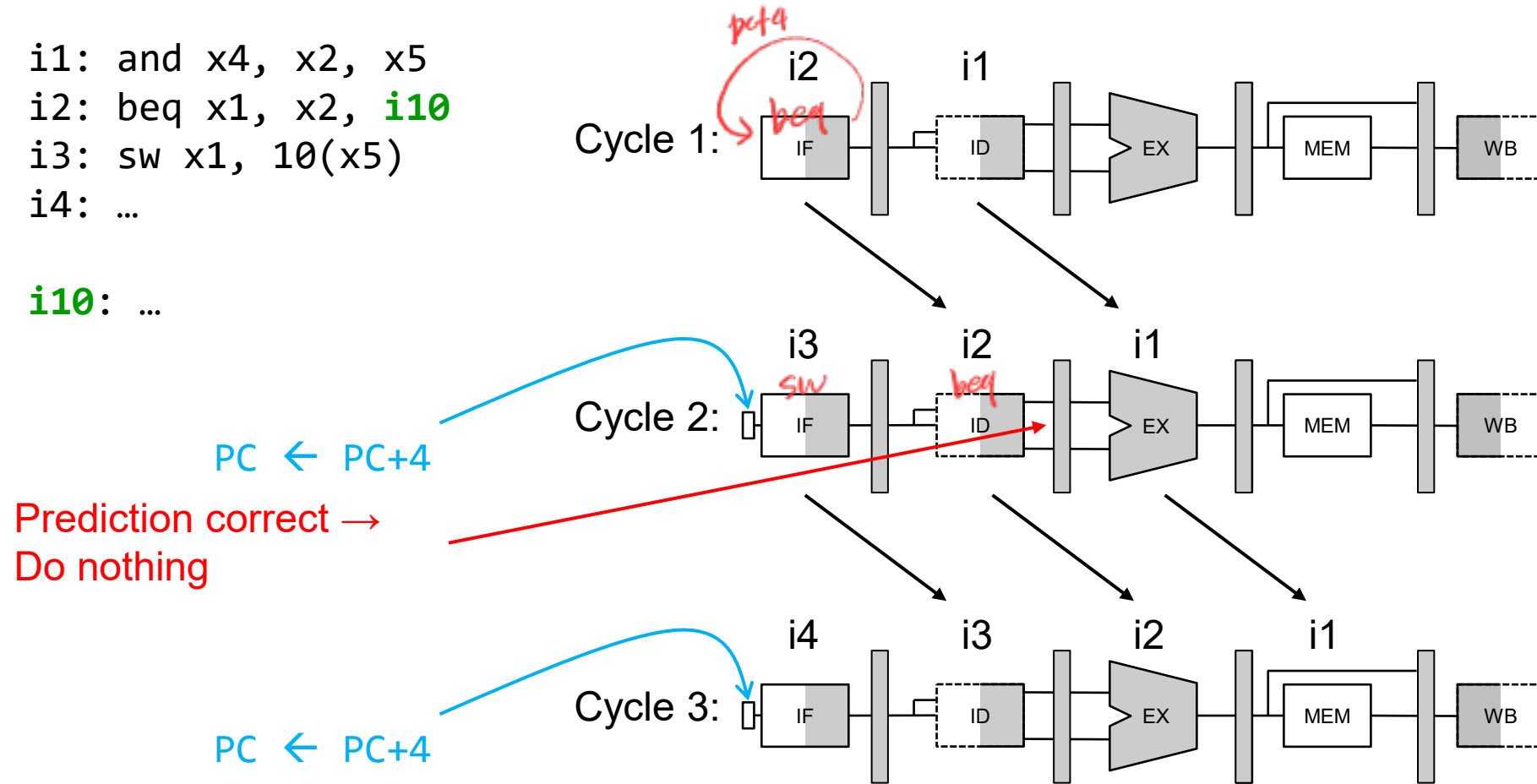
Prediction was incorrect



Branch Prediction: Always Not Taken

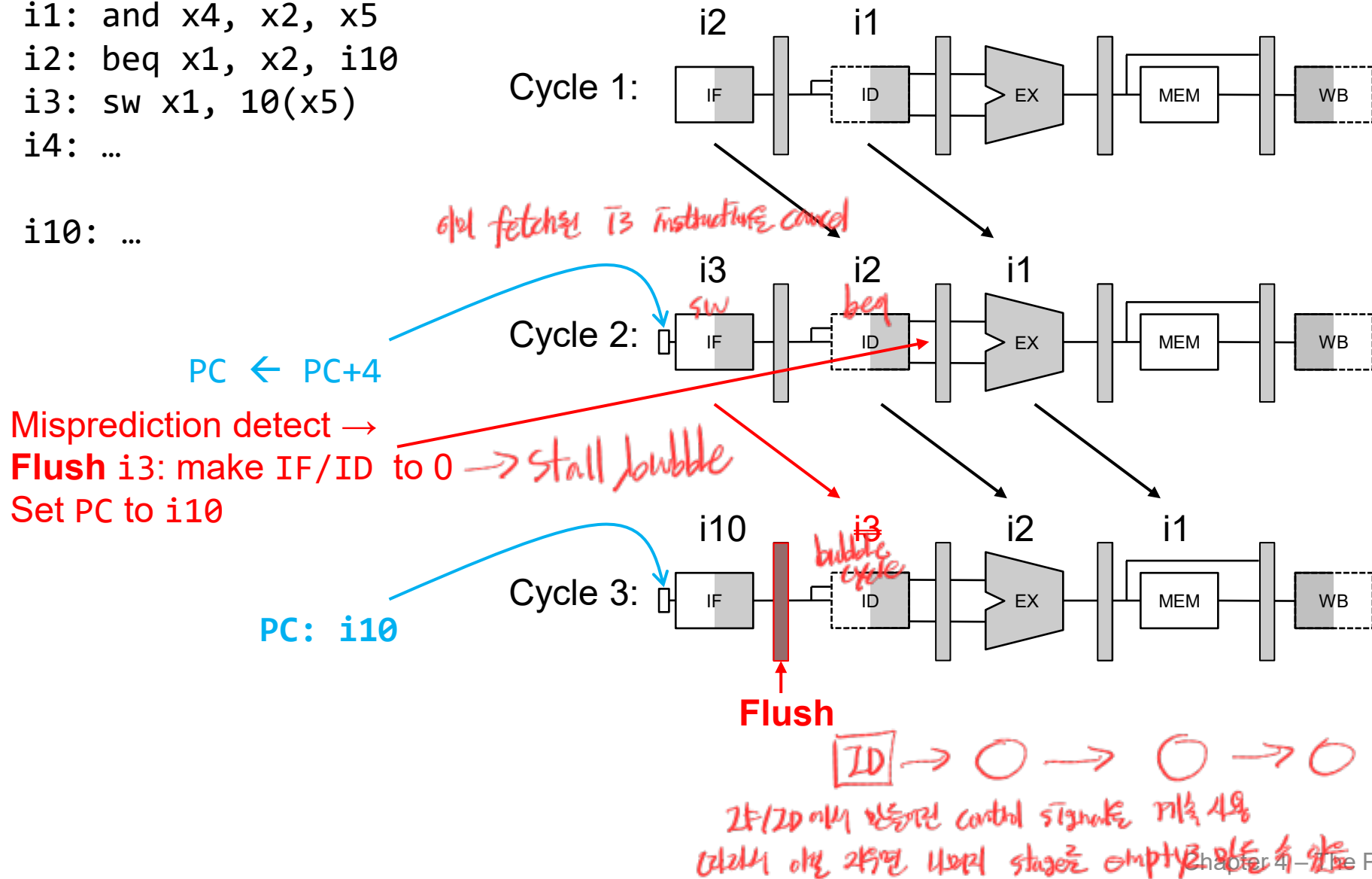
i1: and x4, x2, x5
i2: beq x1, x2, **i10**
i3: sw x1, 10(x5)
i4: ...

i10: ...

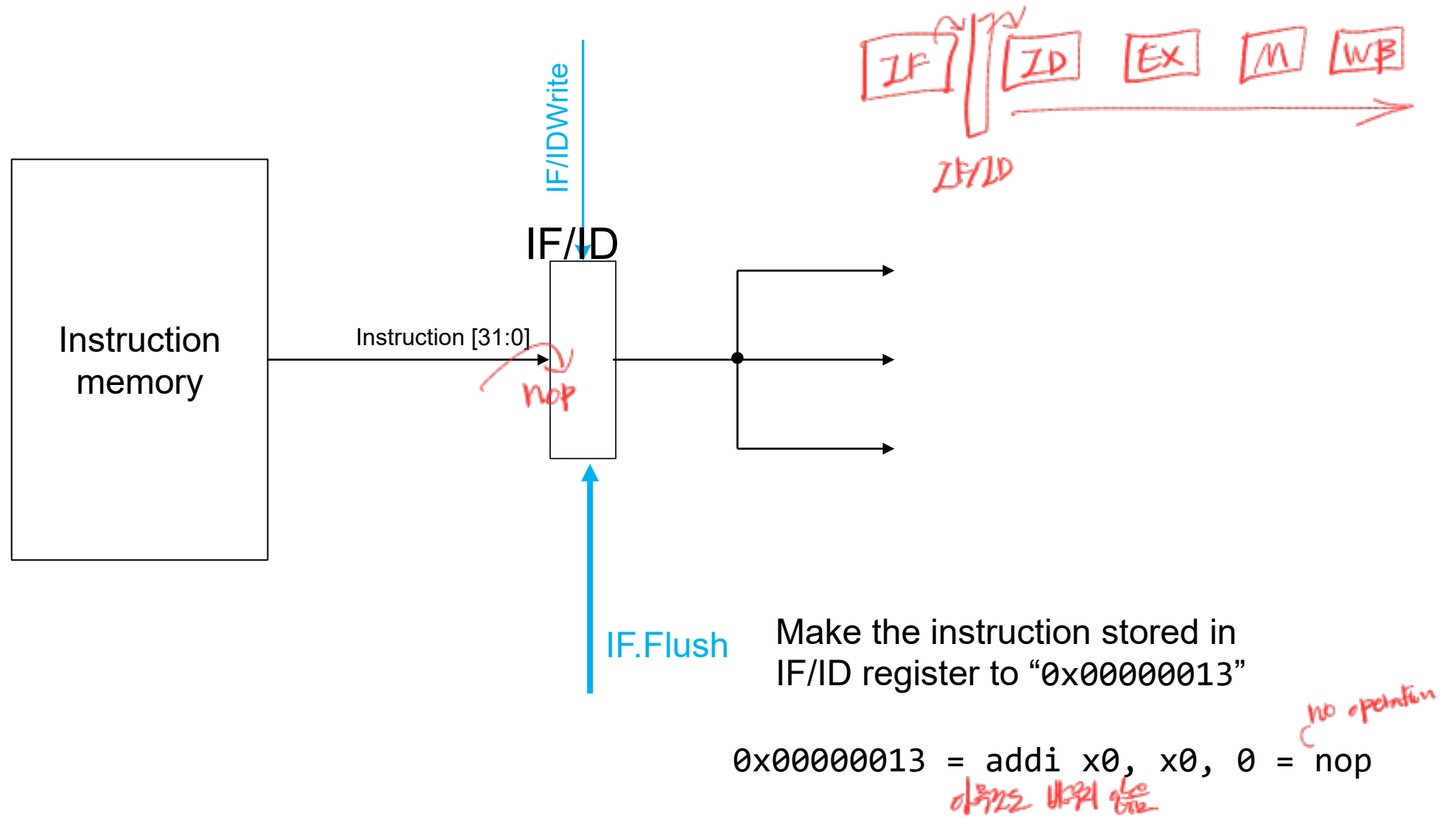


Flushing Mispredicted Branch Result

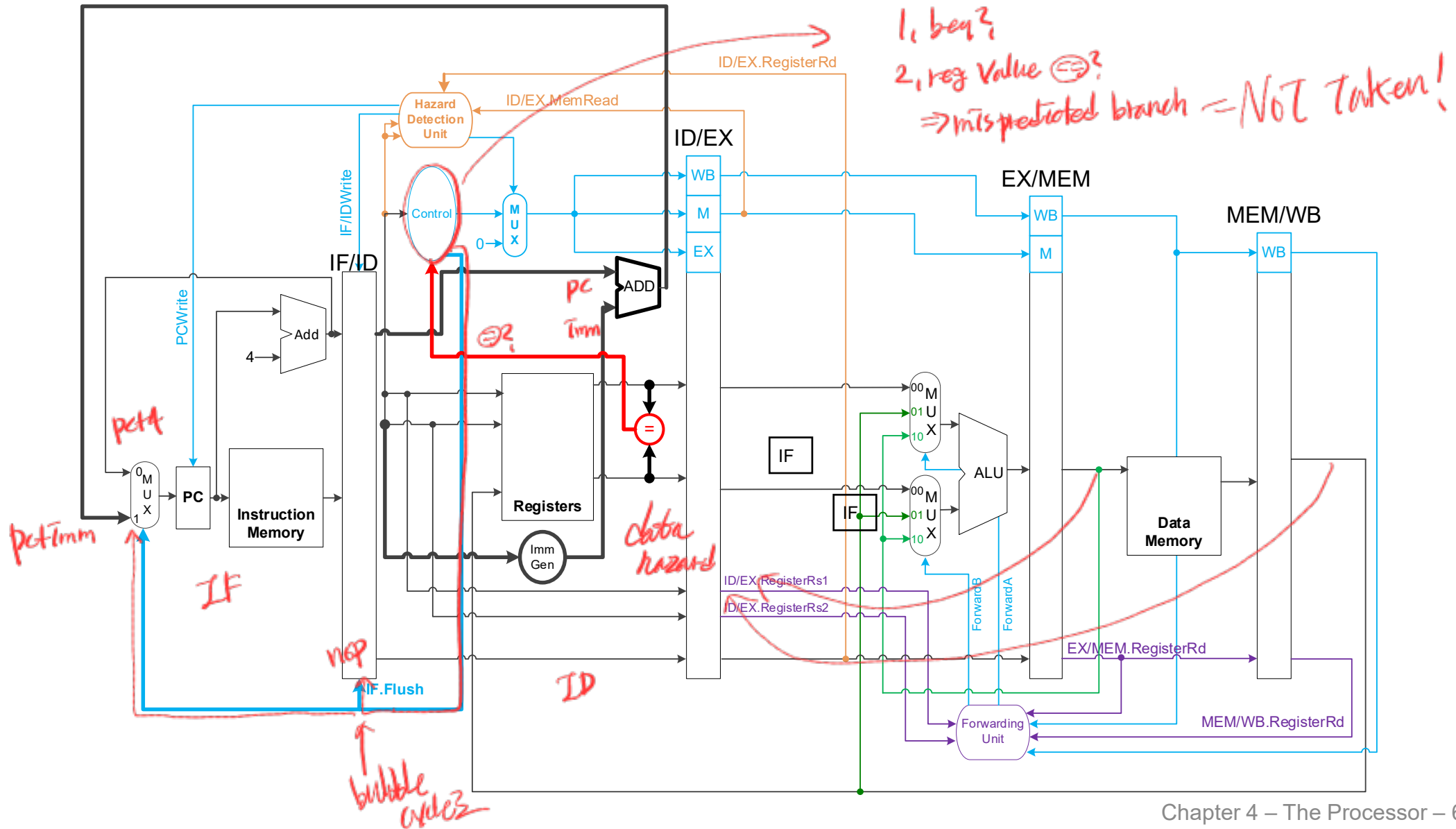
i1: and x4, x2, x5
 i2: beq x1, x2, i10
 i3: sw x1, 10(x5)
 i4: ...
 i10: ...



Adding a Bubble While Canceling the Inst.



Pipeline with Mispredicted Branch Flushing



More-Realistic Branch Prediction

여기서 static prediction
다음은 static과 dynamic

■ Static branch prediction *Always taken or not taken*

- ❖ Based on typical branch behavior
- ❖ Example: loop and if-statement branches

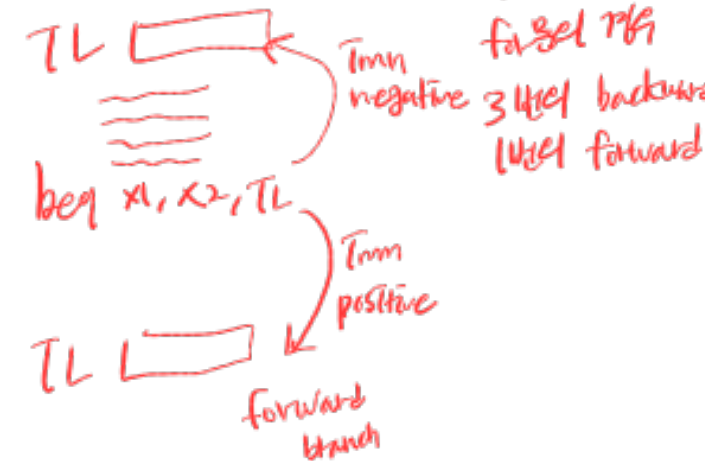
- Backward branches → Predict to be **taken**
 - Forward branches → Predict to be **not taken**
- 가장 많이 더 큰 쪽은 always taken
slightly better than always taken/not-taken*

*loop의 loop
→ 99% success*

*loops for n times
(n-1 : taken branch
1 : not taken branch)*

** compiler technique
add hint for every branch*

*loop의 2, 3, 4번
backward branch*



■ Dynamic branch prediction

- ❖ Assume the future will be similar to the current trend
- ❖ Hardware monitors the current branch behaviors
 - e.g., record the recent history of branch outcomes of each branch instructions
- ❖ Predict the branch outcome based on the recorded information

*Compiler가 static if, loop의
실행 횟수, 더 높은 확률로 taken
하는 것이 hint를 남겨 CPU에
알려 줄
→ 바뀌지 않는 static → dynamic으로
changing하는 effect*

*Not → T
20% → NT*

Dynamic Branch Prediction

- **Branch Prediction Buffer** (a.k.a. branch history table)
- Indexed by recent branch instruction addresses
- Stores outcome (taken/not taken)
binary outcome
- To execute a branch
 - ❖ Check table, expect the same outcome
 - ❖ Start fetching from the fall-through or target
pc+4 *pc+imm*
 - ❖ If wrong, flush pipeline and flip prediction
NT → T

Branch Prediction Buffer

Instruction Memory *→ DRAM*

SW

Address	Instruction
...	...
0x1000	beq
0x1004	lw
0x1008	beq
0x100c	beq
0x1010	add
0x1014	beq
0x1018	nop
0x101c	beq
...	...
...	...

*32 bit
→ 32GB*

*branch의 다음 지점도 →
programmer가 beq의 위치를*

Branch Prediction Buffer *→ within the CPU*

HW

Last branch result
...
0 (not taken)
0
1 (taken)
0
0
1
0
0
...
...

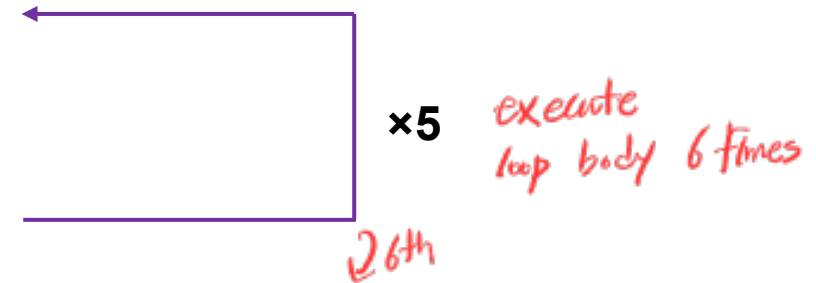
*1 or 2 bit
→ 1GB*

이걸 CPU에 ?

1-Bit Predictor

```

loop:  ...      i1
      ...      i2
      ...      i3
      beq ..., ..., loop  i4
      ...      i5
      ...      i6
  
```



1→2→3→4 → 1→2→3→4 → 1→2→3→4 → 1→2→3→4 → 1→2→3→4 → 1→2→3→4 → 5→6...

BPB entry for i4:
(1-bit)

NT T T T T T NT

Prediction on i4:

NT T T T T T

Branch outcome:

T T T T T NT

Incorrect

Correct

Correct

Correct

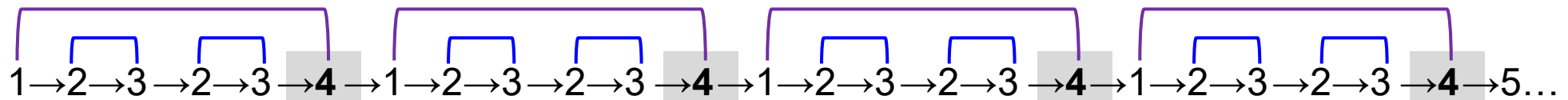
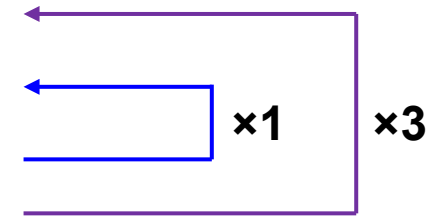
Correct

Incorrect

1-Bit Predictor: Limitations

outer: ...
inner: ...
 beq ..., ..., inner
 beq ..., ..., outer
 ...

i1
i2
i3
i4
i5



BPB entry for **i4**:
(1-bit)

NT

T

T

T

Prediction on **i4**:

NT

T

T

T

Branch outcome:

T

T

T

NT

Incorrect

Correct

Correct

Incorrect

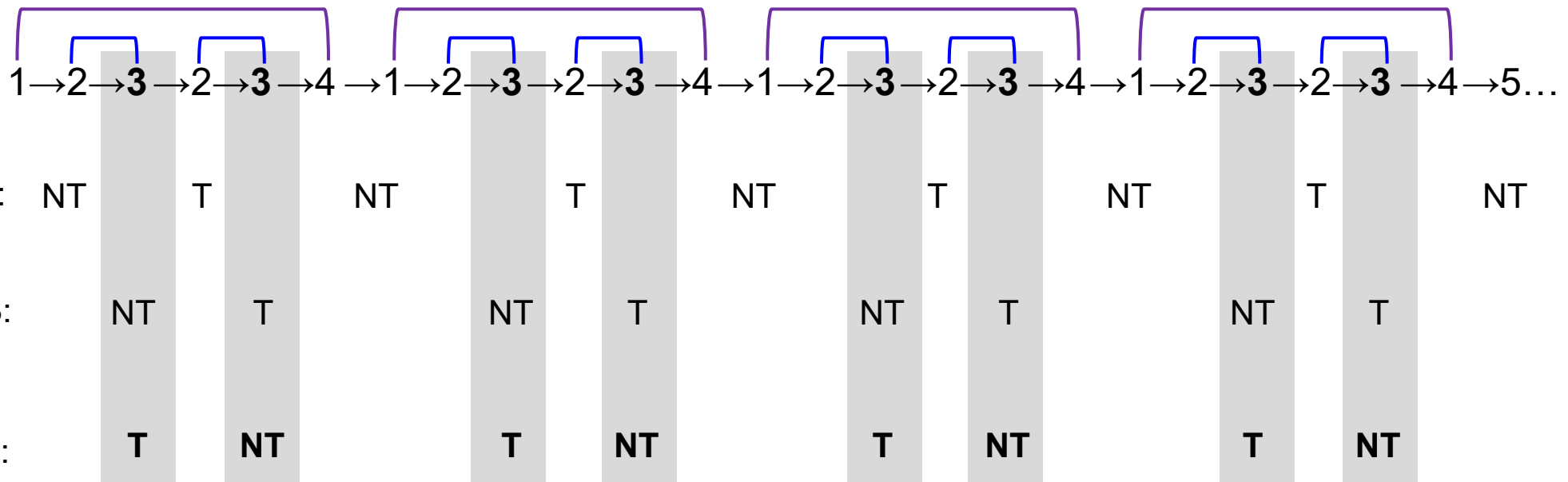
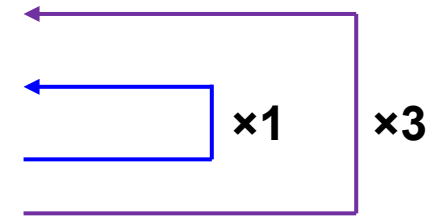
1-Bit Predictor: Limitations

```

outer: ...
inner: ...
    beq ..., ..., inner
    beq ..., ..., outer
    ...

```

i1
i2
i3
i4
i5



Incorrect Incorrect

Incorrect Incorrect

Incorrect Incorrect

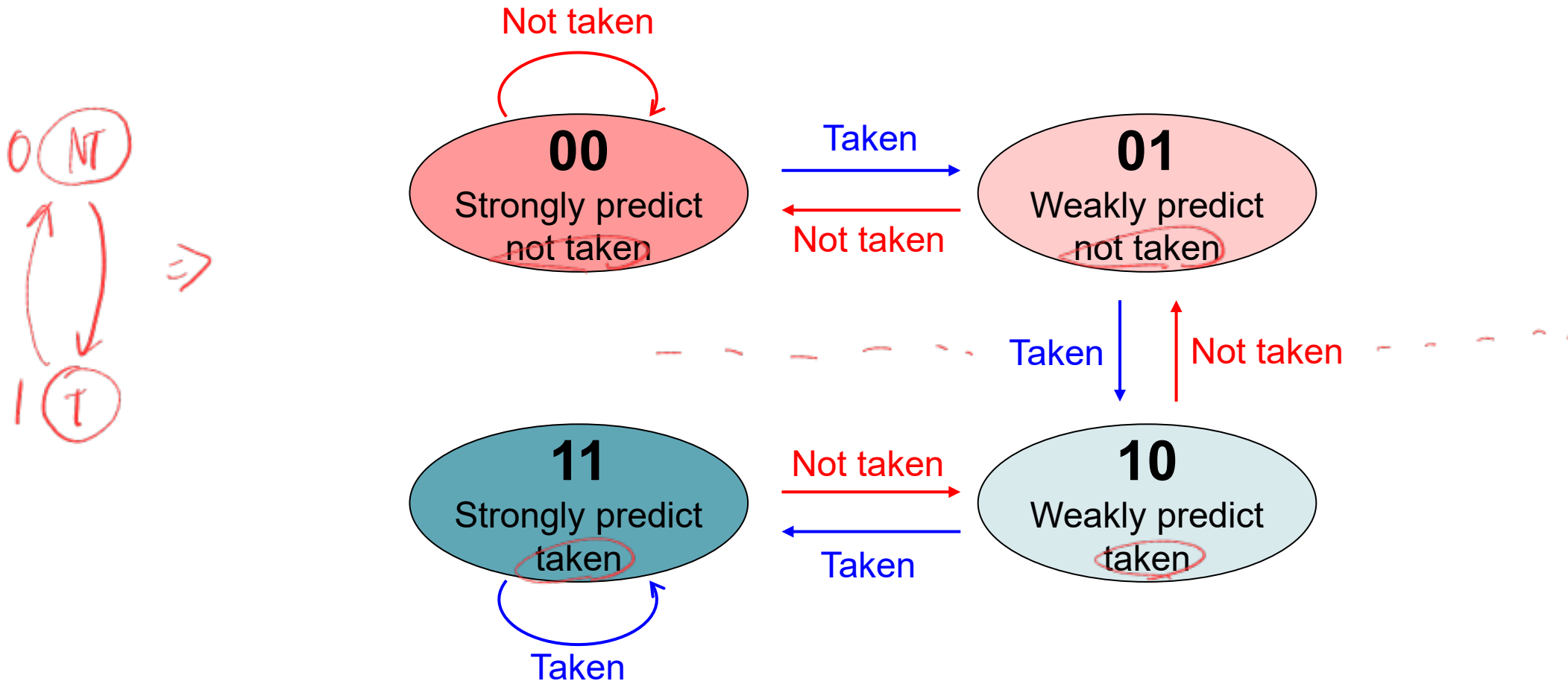
Incorrect Incorrect

T, NT가 번갈아 나오지 않음

2-Bit Predictor

taken/not taken 보라 더 많은 stage 필요

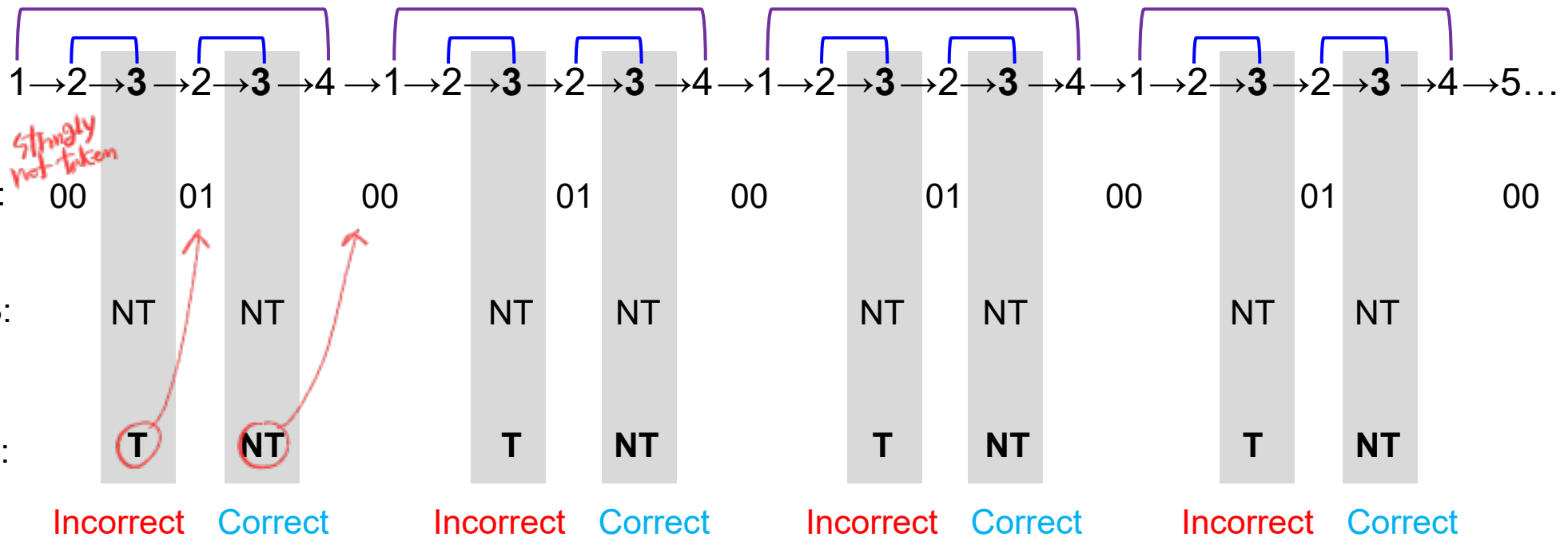
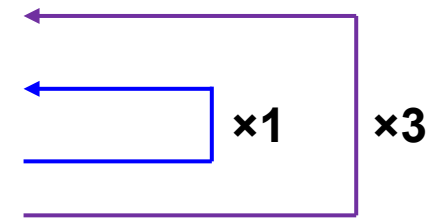
- Only change prediction on two successive mispredictions



2-Bit Predictor

outer: ...
inner: ...
 beq ..., ..., inner
 beq ..., ..., outer
 ...

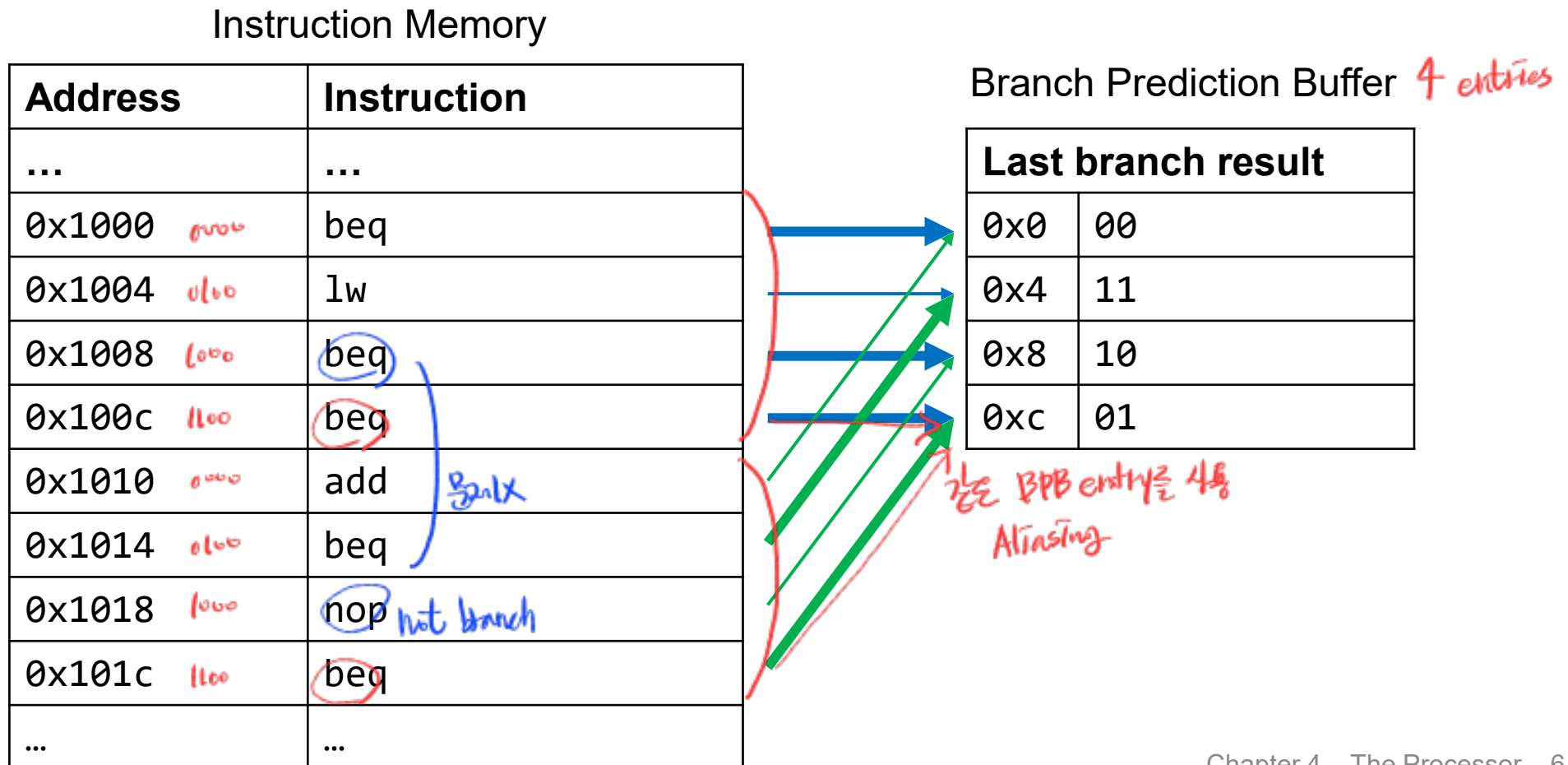
i1
i2
i3
i4
i5



Branch Prediction Buffer

Small, reuse BPB

- $\text{sizeof}(\text{instruction memory}) \gg \text{sizeof}(\text{prediction buffer})$
 - ❖ Multiple instructions are mapped to a single entry
 - ❖ Aliasing (i.e., collision) happens



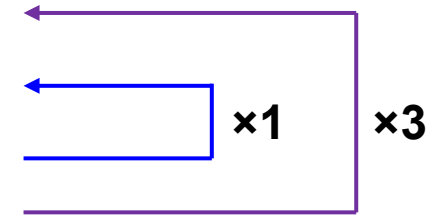
2-Bit Predictor

```

outer: ...
inner: ...
    beq ..., ..., inner
    beq ..., ..., outer
    ...

```

i1
i2
i3
i4
i5

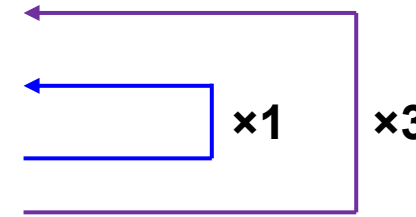


	1→2→ 3 →2→ 3	→	4 →1→2→ 3 →2→ 3	→	4 →1→2→ 3 →2→ 3	→	4 →1→2→ 3 →2→ 3	→	4 →5...				
BPB entry for i3&i4: (2-bit)	00	01	00	01	10	01	10	11	10	11	11	10	01
Prediction:	NT	NT	NT	NT	T	NT	T	T	T	T	T	T	T
Branch outcome:	T	NT	T	T	NT	T	T	NT	T	T	NT	NT	
	I	C	I	I	I	I	C	I	C	C	I	I	

2-Bit Predictor

outer: ...
 inner: ...
 beq ..., ..., inner
 beq ..., ..., outer
 ...

i1
 i2
 i3
 i4
 i5



hybrid branch predictor
 (simpler one not fully warmed up)
 (complex one (fully warmed up) predictor will take over)

the student
 will start

BPB entry for i3&i4:
 (2-bit)

	1	2	3	2	3	→	4	1	2	3	2	3	→	4	1	2	3	2	3	→	4	1	2	3	2	3	→	4	5...
	01		10		01		10		11		10		11		11		11		10		11		11		11		10		01
Prediction:	NT		T		NT		T		T		T		T		T		T		T		T		T		T		T		T
Branch outcome:	T		NT		T		T		NT		T		NT		T		T		NT		T		T		NT		NT		NT
							C				C				C				C		C		C						

warm-up period

Calculating the Branch Target

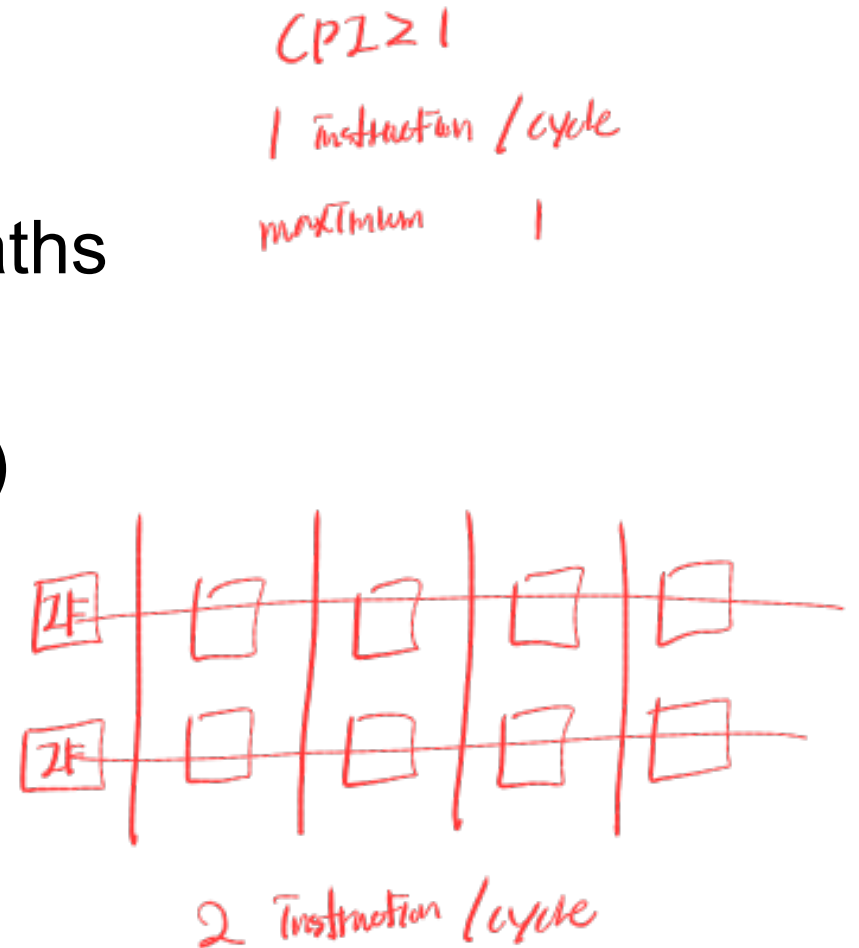
- Even with predictor, still need to calculate the target address
 - ❖ 1-cycle penalty for a taken branch
- **Branch Target Buffer** *BTB*
 - ❖ Cache of target addresses
 - ❖ Indexed by PC when instruction fetched
 - If hit and instruction is branch predicted taken, can fetch target immediately

Correct: execute Instruction
BTB
Incorrect: cancel

Multiple Issue

- Start multiple instructions per clock cycle
- Replicate pipeline stages $\Rightarrow$ multiple datapaths
- CPI < 1 , so use Instructions Per Cycle (IPC)
 - ❖ E.g., 4-way multiple-issue
 - peak CPI = 0.25, peak IPC = 4
 - But dependencies reduce this in practice

X pipeline hazard



Multiple Issue

■ Static multiple issue

- ❖ **Compiler** groups instructions to be issued together
- ❖ Packages them into “issue slots”
- ❖ Compiler detects and avoids hazards

■ Dynamic multiple issue

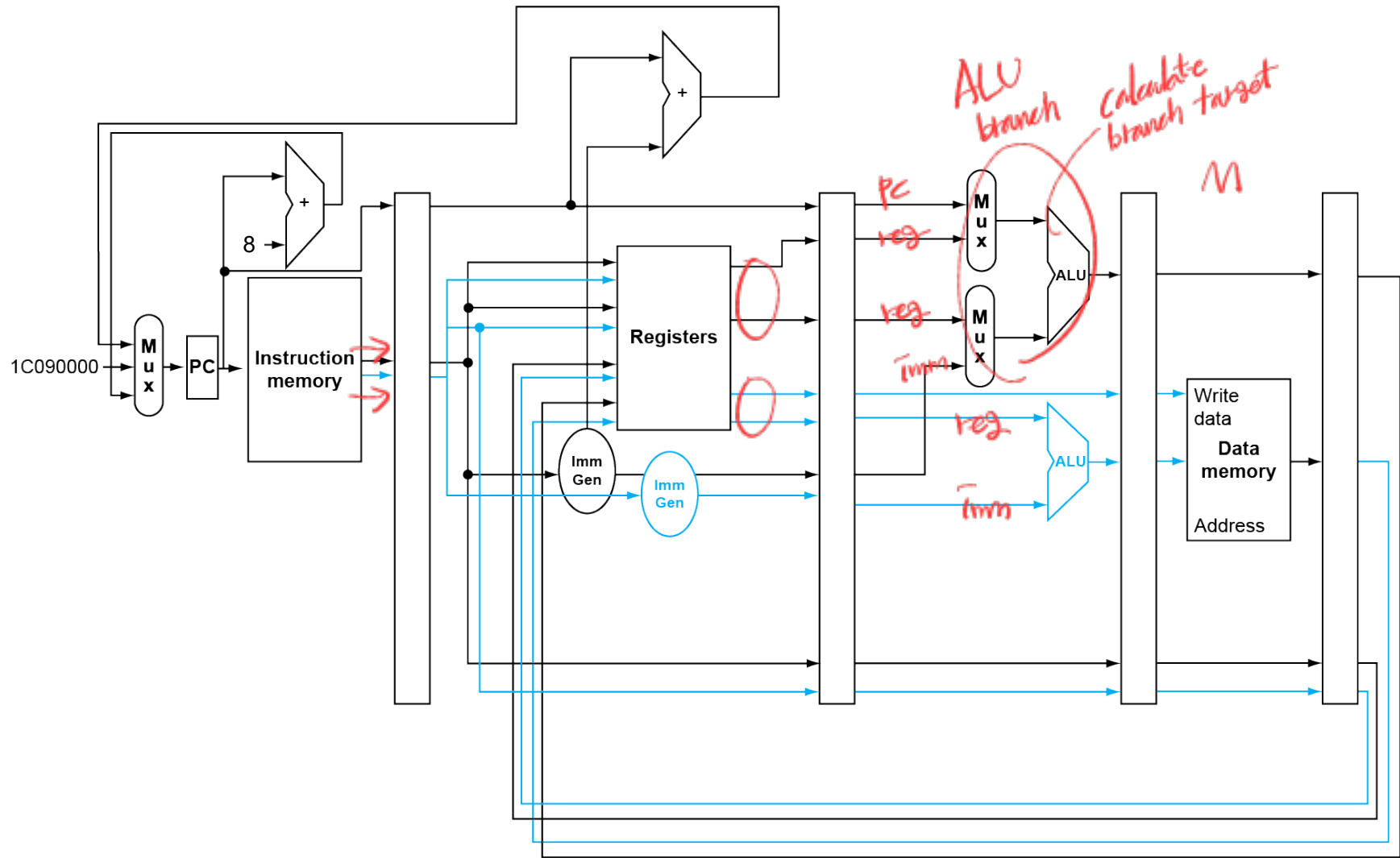
- ❖ **CPU** examines instruction stream and chooses instructions to issue each cycle
- ❖ Compiler can help by reordering instructions
- ❖ CPU resolves hazards using advanced techniques at runtime

RISC-V with Static Dual Issue

- Two-issue packets
 - ❖ ❶ One ALU/branch instruction
 - ❖ ❷ One load/store instruction
 - ❖ 64-bit aligned
 - ALU/branch, then load/store
 - Pad an unused instruction with nop

Address	Instruction type	Pipeline Stages						
n	ALU/branch	IF	ID	EX	MEM	WB		
n + 4	Load/store	IF	ID	EX	MEM	WB		
n + 8	ALU/branch		IF	ID	EX	MEM	WB	
n + 12	Load/store		IF	ID	EX	MEM	WB	
n + 16	ALU/branch			IF	ID	EX	MEM	WB
n + 20	Load/store			IF	ID	EX	MEM	WB

RISC-V with Static Dual Issue



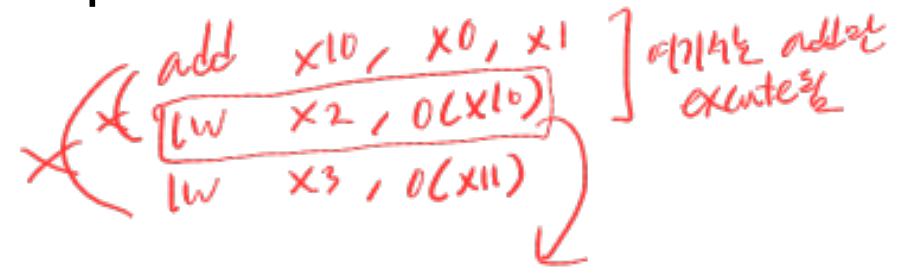
Hazards in the Dual-Issue

■ EX data hazard

- ❖ Forwarding avoided stalls with single-issue
- ❖ Now can't use ALU result in load/store in same packet

```
add  x10, x0, x1  
lw   x2, 0(x10)
```

- ❖ Split into two packets, effectively a stall

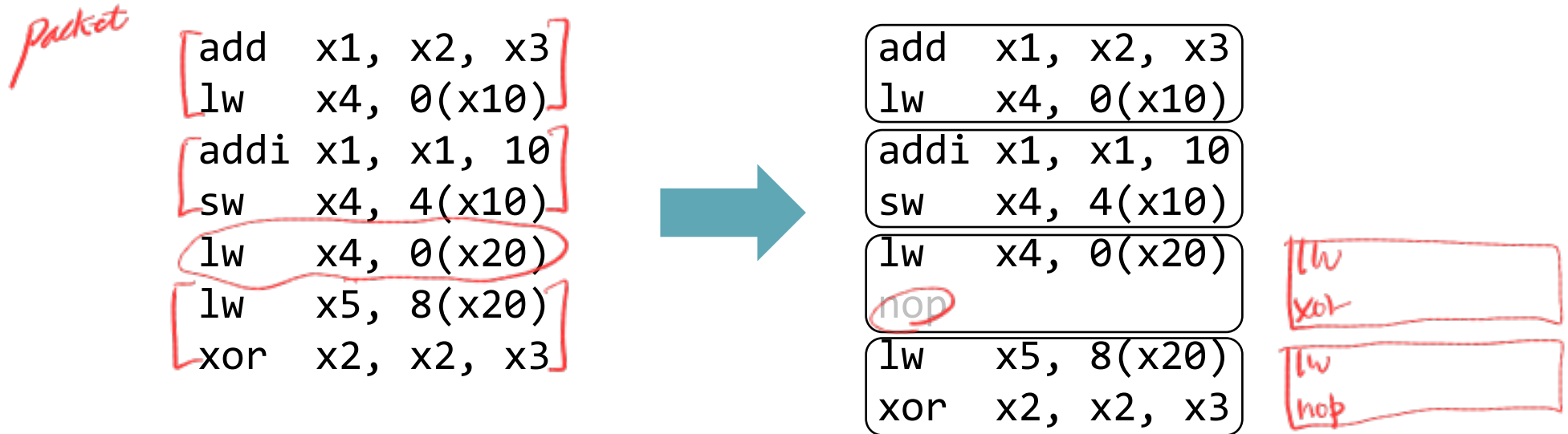


■ Load-use hazard

- ❖ Still one cycle use latency, but now two instructions

Scheduling Example (1)

- Schedule this for dual-issue



	ALU/branch	Load/store	cycle
	add x1, x2, x3	lw x4, 0(x10)	1
	addi x1, x1, 10	sw x4, 4(x10)	2
	nop	lw x4, 0(x20)	3
	xor x2, x2, x3	lw x5, 8(x20)	4

Scheduling Example (2)

■ Schedule this for dual-issue

Loop: lw *x31*, *0(x20)* } # *x31=array element*
add *x31*, *x31*, *x21* } # *add v in x21*
sw *x31*, *0(x20)* } # *store result*
addi *x20*, *x20*, -4 } # *decrement pointer*
bne *x20*, *x0*, Loop } # *branch x20!=0*

	ALU/branch	Load/store	cycle
Loop:	nop	lw <i>x31</i> , <i>0(x20)</i>	1
	nop	nop	2
	add <i>x31</i> , <i>x31</i> , <i>x21</i>	nop	3
	addi <i>x20</i> , <i>x20</i> , -4	sw <i>x31</i> , <i>0(x20)</i>	4
	bne <i>x20</i> , <i>x0</i> , Loop	nop	5

bubble

5 packet

Scheduling Example (3)

Compiler가 더 똑똑한 편

■ Schedule this for dual-issue

```

Loop: lw    x31, 0(x20)    # x31=array element
      addi  x20, x20, -4   # decrement pointer
      add   x31, x31, x21  # add v in x21
      sw    x31, 4(x20)    # store result
      bne   x20, x0, Loop  # branch x20!=0
  
```

Handwritten annotations: A red arrow points from the `sw` instruction back to the `add` instruction. A red circle highlights the `4` in `4(x20)`. A red bracket groups the last three instructions.

	ALU/branch	Load/store	cycle
Loop:	addi x20, x20, -4	lw x31, 0(x20)	1
	nop	nop	2
	add x31, x31, x21	nop	3
	bne x20, x0, Loop	sw x31, 4(x20)	4

5
↓
4 packet

Dynamic Multiple Issue

Compiler가 미리 다는걸 불가능
CPU가 더 복잡하게 결정해야 함...

- “Superscalar” processors ←
- CPU decides whether to issue 0, 1, 2, ... each cycle
 - ❖ Avoiding structural and data hazards
- Avoids the need for compiler scheduling
 - ❖ Though it may still help
 - ❖ Code semantics ensured by the CPU